FAQ - FRCA Election App

A plain-language FAQ covering the questions that come up most often, grouped by who's asking. Where a topic deserves a deeper answer, this page links to the canonical document instead of duplicating.

- For brothers in the pew - see Voters
- For elders, the chairman, the secretary - see Council & chairman
- For another congregation considering adoption - see Prospective adopting churches
- For "what if the admin / developer isn't around?" - see Continuity & support
- For "is the count trustworthy?" - see Security & auditability

---

Voters

Is my vote anonymous?

Yes. The code you receive at the door is unrelated to the attendance register; code slips are handed out from a shuffled stack. After you submit, the database has no link between your code and your vote. See SECURITY.md - How anonymity is preserved for the technical detail.

Can someone trace my vote back to me?

No. The vote table has no foreign key, column, or field connecting it to the code table. Even an administrator with full database access cannot reconstruct who voted for whom. The only link the system tracks is "this code was used", not "this code voted for X".

What if I vote on my phone AND hand in a paper ballot?

Don't. The app warns about this on the splash and ballot pages with a red banner. Choose ONE method only. If both come in, the result is a ballot anomaly that the chairman will see and have to resolve.

What if I lose my code slip?

Tell the task team. They will issue a new slip from the spare stack and the lost slip should be returned (or, if not returned, noted by the chairman). Voting codes are six characters long and case-insensitive.

What if my phone won't connect to the WiFi?

Use the paper ballot instead. The dual-sided card you received has a paper ballot on one side and the code/QR on the other. Pen, mark, hand in. The system counts paper and digital together.

What if I make a mistake on the ballot before submitting?

Just untick and re-tick. Nothing is recorded until you press Cast Your Vote. After submission your code is burned and the choice is final (Article 12).

Can I see the result on my own phone?

Yes. After voting, your phone automatically shows the live ballot progress (and, once the chairman moves to Final Results, the elected brothers). It mirrors what the projector shows the congregation. No extra navigation needed - it's the same URL you arrived at.

What about brothers without smartphones?

Paper ballots are always available. The dual-sided card lets every brother choose which side to use. The app is a convenience; paper is the standing fallback.

Why two rounds?

If after the first round any vacancy is unfilled (no candidate cleared both Article 6a and 6b thresholds, or fewer candidates cleared than there are vacancies), the council holds another round per Article 7. Candidates the council carries forward appear on round 2; the remainder are dropped.

What's a spoilt ballot?

A paper ballot that can't be counted - too many ticks, no ticks, unclear marks, etc. The chairman records the count of spoilt ballots per office and the system uses that number when calculating the Article 6a threshold.

What if there's a tie?

The Election Rules don't prescribe an automatic tiebreaker. The app reports the tie; the council decides how to resolve it (typically another vote between the tied candidates, or a draw of lots per Article 9 in some traditions). See ELECTION_RULES.md.

Council & chairman

What if the laptop crashes mid-election?

Restart the laptop, restart the app (`python app.py`). All data is preserved in `data/frca_election.db` between restarts - votes, attendance, codes, settings, everything. If the laptop is unrecoverable, hand out paper ballots immediately and start the round over on paper. See FAILSAFE.md.

How do I know the count is correct?

Three layers of verification:

1. **Per-candidate vote totals** are visible on the Voting and Count steps. The "Round N tally" table shows digital + paper + postal per candidate with Article 6a/6b ticks.
2. **Voter audit log** (link from the Voting step) shows every code attempt - accepted, rejected, used twice, etc. - with timestamps.
3. **Database is auditable**: `data/frca_election.db` is a plain SQLite file. The council can inspect every row directly with any SQLite client.

What if a brother insists his vote wasn't counted?

Open the Voter audit log. Search for his code. The log records every attempt that code made (accepted, rejected_invalid, rejected_used, etc.) along with the timestamp. If `vote_submitted` is logged for that code, the vote was counted. If `rejected_already_used` appears twice, two scans of the same QR slip - benign or worth investigating.

Can I see who voted for whom?

No. By design. See the Voters - anonymity section above and SECURITY.md. The database has no link from code to vote.

How do I handle a ballot anomaly (more ballots than participants)?

The Voting and Count steps surface a red anomaly banner when total ballots > brothers participating. Likely causes are: attendance miscount, or a brother voted online AND handed in a paper ballot. The chairman investigates and decides how to record it in the minutes; the app does not auto-resolve.

Can I undo a closed round?

There's a soft-reset action that wipes the current round's paper counts and votes (gated by typing the election name + admin password). Hard-reset clears the entire election. Don't use these casually - they're for recovery from genuine errors. The audit log records the reset.

What if our council interprets a rule differently?

Fork the repo and adjust `voting-app/election_rules.py`

- the rule arithmetic is isolated to one file. Update the matching tests in `tests/test_rules_compliance.py` to pin your interpretation. The README and ELECTION_RULES.md call this out explicitly.

How do I generate the meeting minutes?

After finalising the election, the Final results step has a Download Election Minutes button. The DOCX has placeholders for chairman name, scripture reading, and so on, plus a round-by-round narrative populated automatically.

Can I run a dry run beforehand?

Yes. Use `scripts/seed_demo.py` to populate a sample election with codes and members, then walk the wizard end-to-end. Wipe the database afterwards. See SETUP.md for the dry-run procedure.

What's the role of the chairman vs. the task team?

The chairman drives the wizard sidebar and announces phases. The task team hands out cards, collects paper ballots, helps voters who get stuck. Both follow the USER_GUIDE.md. Either can sit at the laptop; only one person should drive the wizard at a time.

Prospective adopting churches

Is this a commercial product?

No. It's open source under the MIT licence, built for one congregation and shared freely. There's no help desk, no SLA, no warranty. See the disclaimer in the README.

How much does it cost?

Nothing for the software. You'll need a laptop and a WiFi router (roughly AUD 100-200 for an entry-level travel router). No internet fees - the app runs entirely on a local network.

What hardware do we need?

- A laptop running Windows, macOS, or Linux with Python 3.10+
- A WiFi router for the church hall (the brothers' phones connect to it; no internet needed)
- A projector or large display for the live tally
- Optionally a printer for code slips and paper ballots

SETUP.md walks the full hardware list.

Will this work for our Election Rules?

The app encodes one congregation's interpretation of the FRCA Election Rules (Articles 1-13). Other congregations may interpret some articles differently - particularly Article 6a's definition of "valid votes cast", and Article 9's tiebreaker. Read ELECTION_RULES.md to see the article-by-article reading and decision provenance. If you'd interpret things differently, fork and adjust `election_rules.py`.

Who supports it?

Nobody. There is no maintainer with an obligation to respond. The author welcomes feedback and contributions on GitHub but makes no commitment to act on them. If your congregation depends on supported software, consider a commercial voting platform (e.g. ElectionBuddy) or stay with paper.

What if our IT skills are limited?

You will need at least one volunteer comfortable with Python on the command line, basic networking, and the concept of "edit a config file". They don't need to be a software developer. The setup is documented step-by-step but does involve a real install on a real laptop. If no one in the congregation is willing to be that person, this software is probably not the right fit.

Can we run on someone else's server?

Not as designed. The app is built around running on a single laptop on a local WiFi network with no internet. That model is deliberate: no third-party data exposure, predictable latency, runs in a power cut on the laptop battery. You could in principle host it on a cloud server but you'd lose those guarantees and the security model assumes a trusted local network.

How long does setup take?

Plan a full evening to install Python, run the app, do a dry run with a few volunteers' phones, and verify your printer prints code slips and ballots correctly. Plan another evening close to the actual election to do a full dry run with a realistic member count. See SETUP.md.

What's the licence?

MIT. You can fork, modify, redistribute, run for any purpose, commercial or otherwise, as long as the licence text and copyright notice come along. There is no warranty.

What if there's a bug?

Report it on GitHub. The maintainer might fix it, might not. You have the source code and the right to fix it yourself. For election-day fallback, paper ballots are always available - that's the whole point of the dual-sided card.

Continuity & support

What happens if the admin / developer isn't available?

The app is designed to be operated by any volunteer who can follow the USER_GUIDE.md on election day. No developer involvement is needed once the laptop is set up and a dry run has been completed. The wizard sidebar walks the chairman through every step in order.

For setup or modifications you'll need someone with basic Python / command-line skills, but that's an annual or one-off task, not an election-day dependency.

What if we hit a bug on election day and the admin / developer isn't reachable?

Two layers of fallback:

1. **Paper ballots are always present.** Every brother gets a dual-sided card with both a paper ballot and a code. If the digital side fails, brothers vote on paper. The chairman manually counts. The election still completes per the Election Rules.
2. **Restart fixes most things.** Restart the laptop, restart the app, the database is preserved. See FAILSAFE.md.

The principle is: the app is a convenience that speeds up counting, not a single point of failure for the election.

Can our own task team learn to maintain it?

Yes. The codebase is a single Flask app (`voting-app/app.py`) plus templates, with a documented database schema, a passing test suite (237 tests), and the rule arithmetic isolated to `election_rules.py`. Anyone with intermediate Python familiarity and a few evenings to read the codebase can take over maintenance.

Where do we go for help if the admin / developer is unreachable?

In order:

1. The relevant documentation in `voting-app/docs/` - this FAQ, USER_GUIDE.md, ADMIN_GUIDE.md, FAILSAFE.md, SETUP.md.
2. The test suite (`pytest voting-app/tests/`) tells you what behaviour is expected and pins it.
3. GitHub Issues on the repo - others may have hit the same problem.
4. Hand out paper ballots and run the election the traditional way.

Security & auditability

How do I know the admin can't change votes?

You don't, in absolute terms - and neither can you with paper-only ballots, where the chairman or counting team has the same theoretical ability. The mitigation is the same in both cases: a trusted council, multiple witnesses, and an auditable record. For the digital side specifically:

- The database is a plain SQLite file the council can inspect at any time, before, during, and after the election.
- The Voter audit log records every vote-related action with timestamp and result.
- Source code is open and inspectable. Any deviation from the documented behaviour would be visible in the diff.

See SECURITY.md for the threat model.

Could someone hack the system from outside?

The system runs on a local WiFi network with no internet exposure. Outside attackers can't reach it. An attacker would need to be physically on the church WiFi, and even then they'd hit the same authentication checks (valid code required) as a regular voter. The realistic threat is "curious brother", not "remote adversary".

Could a brother in the room try to game it?

Possible attacks:

- **Guess a code.** Codes are 6 random characters from a 31-char alphabet (uppercase + digits, no O/0/I/1/L). One in ~887 million per attempt; a guesser would be detected and locked out.
- **Reuse a slip.** Codes are burned on first use. A second attempt with the same code is rejected and logged.
- **Vote on someone else's phone.** Same threat as paper, where anyone can fill in a paper ballot and post it. Mitigated by the task team handing one card per brother at the door against the attendance register.
- **Vote twice (paper + digital).** The system flags ballot anomalies (more ballots than participants) on the projector for the chairman to investigate.

Are the codes truly random?

Yes. Generated with Python's `secrets` module (cryptographically strong RNG), 6 characters from a 31-char alphabet, deduplicated against the codes table. Plaintext codes are stored hashed (`hash_code()`); the only time plaintext is visible is on the Codes step immediately after generation.

What about data retention?

The database lives in `data/frca_election.db` on the laptop. Nothing leaves the local network. After the election the council can:

- Archive the database (copy to a safe location for the records)
- Wipe the database to start fresh for the next election
- Both - keep an archive AND wipe the live copy

The minutes DOCX is the formal record; the database is the audit record.

What if the laptop is stolen?

The database contains the election's vote totals (which are public once announced anyway), member names, and hashed codes. It does NOT contain plaintext codes (those exist only briefly during the codes step) or any link from voters to votes. So a stolen laptop after the election doesn't compromise anonymity. Before the election it contains generated plaintext codes if not yet printed - keep the laptop secured before printing day.

Is the source code inspectable?

Yes. Everything is on GitHub under the MIT licence. The council can:

- Read the code (it's plain Python and HTML, no obfuscation, no build step)
- Run the test suite (`pytest voting-app/tests/`) to verify behaviour
- Run a local instance and observe its behaviour against test cases
- Compare what's running on election day against the published source

How do I audit a result after the fact?

1. Cross-check the Voter audit log against the attendance register - number of code-accepted entries should match in-person participants.
2. Cross-check the per-candidate digital tallies against the database directly: `SELECT candidate_id, COUNT(*) FROM votes WHERE round_number = ? GROUP BY candidate_id;`
3. Cross-check paper tallies against the physical paper ballots (kept by the secretary after counting).
4. Cross-check the elected-status calculation against ELECTION_RULES.md by hand.

If any of these don't reconcile, that's a flag worth investigating.

Has this been independently security-reviewed?

No. The threat model is "trusted community of brothers in a church hall" - not the kind of environment that justifies a paid security review. If your congregation has different threat assumptions, get your own review or use a commercial product.